

Compilation Is Not Semantic Correctness: An Agentic Rerun and Evaluation Audit of AlgoVeri’s Lean Benchmark

Joey Li
joeyli.shsid@gmail.com

August 2026

Abstract

AlgoVeri evaluates verified code generation for 77 classical algorithms aligned across Lean, Dafny, and Verus. We independently rerun its Lean track with four direct-API conditions—GPT-5.5, GPT-5.6-sol, and Claude Opus 5 with adaptive thinking enabled or disabled—and a separate compiler-using Claude Code + Opus 5 condition. Complete-output direct conditions compile at rates between 92.2% and 98.7%, yet only 31.2%–57.1% of the full benchmark receive a semantic pass, depending on generator and judge. On a controlled 65-task subset without teacher-provided proof holes, the enhanced coding agent compiles 63/65 and receives 53/65 semantic passes from GPT-5.4 at temperature zero and 55/65 from GPT-5.6-sol at temperature zero, compared with 39/65 and 44/65 for the strongest complete direct condition, respectively.

The rerun also reveals evaluation defects that materially affect interpretation. The `trie_search` scaffold contains a duplicate declaration and undefined identifier outside model-editable regions. Twelve tasks contain teacher-owned `sorry` terms explicitly permitted by the generation prompt, while the semantic prompt broadly instructs the judge to reject `sorry`; GPT-5.4 rejects all 34 compiled direct-model answers observed on these tasks. Judge choice remains consequential even at temperature zero: GPT-5.4 and GPT-5.6-sol disagree on 22 of 334 compiled condition–task pairs (93.4% agreement, Cohen’s $\kappa = 0.868$). Qualitative analysis shows that most compiler-accepted failures use an extensionally correct but wrong algorithm, a specification oracle, brute-force enumeration, or a verified fallback. We release all generated programs, three semantic-judge views, case-level analyses, a local Lean runner, a coding-agent harness, and an interactive audit dashboard. The results support reporting compilation, algorithmic fidelity, output coverage, and judge robustness as separate dimensions rather than treating Lean compilation as a sufficient success criterion.

1 Introduction

Large language models can now generate nontrivial formal programs and proofs, but evaluation is more delicate than ordinary code generation. A Lean file may compile while implementing a different algorithm, delegating to a library theorem, exploiting an underspecified contract, or embedding a nonconstructive oracle. Conversely, a correct student contribution can be penalized for immutable content supplied by the benchmark. These distinctions matter because a compiler answers a syntactic and logical question—whether the complete term type-checks—whereas a benchmark may intend to measure algorithm identity, implementation quality, and proof authorship.

AlgoVeri was introduced to study exactly this verified-code-generation setting on 77 classical algorithms represented in Lean, Dafny, and Verus[11]. Its aligned natural-language tasks and contracts make it a valuable testbed for separating formal verification from semantic algorithm selection. The original paper reported a best Lean result of 7.8%, emphasizing the difficulty of explicit proof construction. Since then, newer reasoning models and tool-using coding agents have made it useful to revisit both capability and measurement validity.

This work makes five contributions:

1. We provide a complete rerun of the Lean track using four direct generation conditions and three semantic judging conditions.
2. We implement and evaluate an isolated Claude Code condition with constrained compiler access, adaptive repair, source ownership enforcement, and final independent verification.
3. We measure output coverage, Lean compilation, semantic acceptance, repair dynamics, costs where available, judge agreement, and paired differences on a common task scope.

4. We audit every compiler failure, every coding-agent semantic rejection, every judge disagreement, and all task-level outcomes. The complete matrix appears in Appendix 5; machine-readable analyses are released with the artifact[4].
5. We identify a malformed scaffold, an ownership contradiction involving teacher-permitted `sorry`, and an asymmetry between the generation and compiler-revision prompts, then propose concrete benchmark fixes.

2 Background and Related Work

2.1 Lean and verified code generation

Lean 4 combines dependent type theory, an extensible theorem prover, and a functional programming language[3]. Mathlib supplies a large, community-built library of definitions, theorems, and tactics[7]. This ecosystem enables machine-checkable proofs but also creates evaluation choices: using Mathlib is necessary for the supplied tasks, whereas calling an existing implementation of the target algorithm may violate a “from scratch” requirement. The benchmark therefore needs a semantic policy in addition to compiler verification.

Formal-reasoning benchmarks such as miniF2F[12] and LeanDojo[9] primarily evaluate theorem proving. Verified program generation additionally requires executable definitions, functional contracts, termination, and algorithm-specific invariants. DafnyBench[5] and the multi-language vericoding benchmark[1] demonstrate the growing interest in this broader setting. AlgoVeri contributes aligned classical algorithm tasks across verification paradigms[11].

2.2 AlgoVeri’s two-stage Lean metric

Each Lean task contains a natural-language description and a scaffold with four model-editable marker regions: `auxcode`, `code`, `lemma`, and `proof`. The response parser copies only those regions into the teacher scaffold. Generated uses of `sorry`, `admit`, axioms, unsafe definitions, and related bypasses are rejected. A local Lean compiler then checks the merged file.

Compilation alone cannot enforce the named algorithm. AlgoVeri therefore applies an LLM semantic filter to the natural-language task and complete merged file. The judge checks whether the intended algorithm was implemented from scratch and whether the answer uses a compiler trick. This is an instance of model-based evaluation, whose scalability comes with known sensitivity to judge and prompt[13]. We retain the benchmark prompt while explicitly measuring that sensitivity.

3 Research Questions

We organize the study around the following questions.

- RQ1** How often do current frontier models return an output that compiles in the AlgoVeri Lean environment?
- RQ2** How large is the gap between Lean compilation and semantic acceptance?
- RQ3** Does a constrained compiler-using coding agent improve end-to-end semantic performance relative to direct API generation?
- RQ4** How stable are semantic conclusions under temperature and judge-model changes?
- RQ5** Which task, prompt, and evaluator defects threaten the validity of the measured score?

4 Methodology

4.1 Task scopes

The full scope contains all 77 Lean tasks in the public AlgoVeri repository [10]. Four direct conditions target the full scope. The adaptive- thinking Opus run produced only 49 saved outputs; missing outputs remain failures under the end-to-end denominator and are also reported conditionally.

The enhanced coding-agent experiment uses a deterministic source-aware filter that excludes the 12 tasks containing teacher-owned `sorry` outside all four editable regions. Its scope is therefore 65 tasks. We evaluate

every generator on this same 65-task set for fair comparisons. No generation result is selected or discarded based on its later semantic verdict.

4.2 Direct generation conditions

We evaluate GPT-5.5, GPT-5.6-sol, and Claude Opus 5 through the same AlgoVeri chat-and-compiler-revision loop. Each task receives one independent pass, up to 15 generation or repair attempts, and compiler error feedback after an invalid attempt. The OpenAI conditions use medium reasoning and a 32,768-token maximum output. Opus is evaluated both with adaptive thinking and with thinking disabled. Provider-specific reasoning settings are not assumed to be equivalent.

The initial prompt explains the algorithm requirement and prohibits compiler bypasses. After a failed attempt, the stock Lean revision prompt asks for a complete standalone file that addresses the compiler errors. Section 7.3 discusses the semantic information lost in this shorter revision instruction.

4.3 Enhanced coding-agent condition

The coding-agent condition uses Claude Code 2.1.220 with Opus 5, medium effort, and an isolated workspace per task. The agent may read task files, edit only `Solution.lean`, and invoke only a constrained `./check.sh`; network, Git, package installation, and arbitrary shell commands are blocked. Candidate content is copied only from the four editable regions into the original scaffold. The harness then rejects prohibited terms and independently recompiles the merged result.

The enhanced protocol provides an initial budget of USD 2. If independent compilation fails, one fresh compiler-repair session receives the exact verifier feedback and up to another USD 2 while preserving the workspace. The prompt explicitly prohibits alternative algorithms, specification oracles, brute-force enumeration, sorting as a substitute, `Nat.find`, `Classical.choose`, and `decide` over the target property. It also requires an early Lean check and a final requirement-by-requirement semantic audit. The final saved result is always independently checked; the agent’s claim of success is not trusted.

Before this enhanced run, two targeted pilot batches covered 10 and 20 cases that prior direct conditions had failed. Those pilots are reported separately because their selection and prompt differ from the final 65-task condition.

4.4 Lean environment

All candidates are compiled locally without Apptainer. The runner places a temporary file inside a pinned Lake project and invokes `lake env lean`. The environment uses Lean 4.25.0-rc2 (commit 744f9806), Lake 5.0.0, and Mathlib tag v4.25.0-rc2 (commit 766e19e7). The host runs Ubuntu 22.04.5 LTS on an AMD Ryzen Threadripper PRO 7985WX (64 physical cores, 128 threads) with 502 GiB RAM. The visible display adapter is AMD/ATI; generation and semantic inference use remote APIs, so local GPU performance is not part of the treatment.

4.5 Semantic judges

Every saved generation output is evaluated with the same AlgoVeri semantic prompt under three conditions:

1. GPT-5.4, temperature 1 (the initial rerun configuration);
2. GPT-5.4, temperature 0;
3. GPT-5.6-sol, temperature 0 and reasoning effort `none`.

GPT-5.6-sol rejects an explicit temperature when reasoning is enabled in this API, so `none` is required to isolate temperature zero. The temperature-zero outputs are stored in distinct directories and do not overwrite the initial judgments. Up to five turns are allowed only to repair malformed `<analysis>` and `<conclusion>` formatting; this is not a vote across independent samples.

4.6 Metrics and analysis

We report four mutually exclusive observable states: missing output, saved output that does not compile, compiled output rejected by the semantic judge, and compiled output accepted by the judge. “Semantic pass” always means

Table 1: Native-scope results. Semantic columns show pass counts for GPT-5.4 at temperature 1 (5.4-T1), GPT-5.4 at temperature 0 (5.4-T0), and GPT-5.6-sol at temperature 0 (5.6-T0). Percentages use the native scope, not only returned outputs.

Generation condition	Output	Lean compile	5.4-T1	5.4-T0	5.6-T0
GPT-5.5	77/77	75/77 (97.4%)	24/77 (31.2%)	24/77 (31.2%)	24/77 (31.2%)
GPT-5.6-sol	77/77	71/77 (92.2%)	25/77 (32.5%)	27/77 (35.1%)	30/77 (39.0%)
Opus 5, thinking	49/77	49/77 (63.6%)	29/77 (37.7%)	29/77 (37.7%)	31/77 (40.3%)
Opus 5, no thinking	77/77	76/77 (98.7%)	36/77 (46.8%)	39/77 (50.6%)	44/77 (57.1%)
Claude Code + Opus 5	65/65	63/65 (96.9%)	53/65 (81.5%)	53/65 (81.5%)	55/65 (84.6%)

Table 2: Common 65-task scope with no teacher-owned `sorry`. This table should be used for comparisons with the enhanced coding agent.

Generation condition	Output	Lean compile	5.4-T1	5.4-T0	5.6-T0
GPT-5.5	65	65	24 (36.9%)	24 (36.9%)	24 (36.9%)
GPT-5.6-sol	65	60	25 (38.5%)	27 (41.5%)	29 (44.6%)
Opus 5, thinking	47	47	29 (44.6%)	29 (44.6%)	31 (47.7%)
Opus 5, no thinking	65	65	36 (55.4%)	39 (60.0%)	44 (67.7%)
Claude Code + Opus 5	65	63	53 (81.5%)	53 (81.5%)	55 (84.6%)

the conjunction of compiler success, a parsed judgment, and a **YES** verdict. Rates use the stated native or common scope as denominator; we additionally show output-conditional rates where missingness is material.

Judge agreement is measured only on compiled outputs evaluated by both judges. We report raw agreement and Cohen’s κ [2]. Paired agent/direct comparisons on the same 65 tasks use the exact two-sided McNemar test [6]; these descriptive p -values do not make the fixed task suite a random population. Selected binomial intervals are 95% Wilson intervals [8]. All aggregates are regenerated from JSON by `scripts/analyze_rerun_results.py`; no table was transcribed from terminal logs.

5 Results

5.1 Output coverage and compilation

Table 1 answers RQ1: when an answer is returned, compilation is high, but output coverage is itself a first-class failure mode. GPT-5.5 compiles 75/77 (97.4%; Wilson 95% CI [91.0%, 99.3%]), GPT-5.6-sol 71/77 (92.2%), and no-thinking Opus 76/77 (98.7%). All 49 saved adaptive-thinking Opus outputs compile, but 28 tasks have no output, reducing end-to-end compilation to 63.6%. In its final one-attempt sweep over the missing set, 24 requests exhausted 32,768 output tokens with thinking blocks only and four ended in provider overload.

Disabling Opus thinking eliminates this coverage failure and is not offset by worse quality on the matched 49 returned tasks. On those tasks, thinking versus no-thinking semantic passes are 29 versus 30 (5.4-T1), 29 versus 31 (5.4-T0), and 31 versus 35 (5.6-T0). Thus this run provides no evidence that adaptive thinking improves the matched outcome, while it substantially reduces completion reliability.

5.2 Compilation is a weak proxy for semantic success

The central result for RQ2 is the size of the compilation–semantic gap. For example, GPT-5.5 compiles 75 tasks but all three judges accept only 24. No-thinking Opus compiles 76, while acceptance ranges from 36 to 44. Even conditional on compilation, GPT-5.4 at temperature zero accepts 32.0% of GPT-5.5, 38.0% of GPT-5.6-sol, and 51.3% of no-thinking Opus outputs. The compiler verifies the submitted theorem; it does not establish that the program realizes the natural-language algorithm.

The 65-task scope removes the known teacher-`sorry` confound and makes all final conditions directly comparable (Table 2). GPT-5.5 compiles all 65 yet passes only 24. No-thinking Opus is the strongest direct condition at 39/65 under 5.4-T0 and 44/65 under 5.6-T0.

Table 3: Cumulative compilation success by attempt/check count. Direct runs count LLM generation attempts; Claude Code counts observed `./check.sh` invocations, so the curves are descriptive rather than directly cost-equivalent.

Condition	1	2	3	5	10	15
GPT-5.5	21	43	57	67	75	75
GPT-5.6-sol	19	37	53	63	70	71
Opus, thinking	33	45	49	49	49	49
Opus, no thinking	24	43	58	69	75	76
Claude Code	15	37	45	62	63	63

Table 4: Pooled semantic-judge agreement on compiled outputs. A generation of the same benchmark task is a separate pair; pooled observations are not assumed independent.

Comparison	n	Agree	Rate	κ	Left only	Right only
5.4-T1 vs 5.4-T0	334	321	96.1%	0.922	4	9
5.4-T0 vs 5.6-T0	334	312	93.4%	0.868	5	17

5.3 Coding-agent gains

The enhanced coding agent answers RQ3. It compiles 63/65 and passes 53/65 under both GPT-5.4 conditions and 55/65 under GPT-5.6-sol. Its 5.6-T0 semantic rate is 84.6% (Wilson 95% CI [73.9%, 91.4%]), versus 67.7% ([55.6%, 77.8%]) for no-thinking Opus on the same tasks. The agent has 13 unique passes against two unique passes for no-thinking Opus under 5.6-T0 (exact McNemar $p = 0.0074$). The corresponding discordances are 18–1 under 5.4-T1 ($p = 7.6 \times 10^{-5}$) and 16–2 under 5.4-T0 ($p = 0.0013$).

This is an agentic condition, not a like-for-like model replacement. It receives tool access, multiple turns, persistent files, explicit semantic reminders, and an adaptive budget. The observed gain therefore estimates the whole harness treatment. The final 65-task run costs USD 70.26 in recorded Claude Code charges (mean USD 1.08 per task) and contains 6.24 summed agent-hours; direct API dollar cost and duration were not recorded and cannot be compared.

The earlier targeted pilots compiled 4/10 and 13/20, respectively; combined, 17/30 compiled and 14/30 passed the 5.4-T1 judge at a recorded cost of USD 41.48. After adding early checks, a semantic self-audit, a preserved-workspace repair pass, exact verifier feedback, and final independent verification, the broader enhanced condition reaches 63/65 compilation and 53/65 5.4-T1 semantic success. Because task selection and several settings changed, this is an engineering progression rather than an ablation.

5.4 Repair dynamics

Iterative compiler feedback matters: only 19–24 of 77 full-output direct conditions compile on the first attempt, rising to 63–69 by attempt five. GPT-5.5 gains no further success after attempt ten, and GPT-5.6-sol gains one. The coding agent reaches 62/65 by its fifth observed Lean check and one more by its tenth. These are cumulative success-at-repair curves, not $\text{pass}@k$ estimates from independent samples.

5.5 Judge robustness

Table 4 answers RQ4. Temperature zero does not eliminate evaluator variation. Across all five generation conditions, GPT-5.4 at temperatures 1 and 0 flips 13 of 334 compiled pairs. The two temperature-zero judges flip 22 pairs. GPT-5.6-sol is more permissive in aggregate: 17 GPT-5.6-only passes versus five GPT-5.4-only passes, a net gain of 12.

Per-condition agreement ranges from 90.1% to 96.8% for the two zero-temperature judges. Exact disagreement cases and directional flips appear in Appendix 6. Several are substantively ambiguous. For example, the agent’s `bubble_sort` is accepted at 5.4-T1, rejected at 5.4-T0 as selection-sort-like, and accepted at 5.6-T0. Its `polymul_naive` is rejected by GPT-5.4 for an unused placeholder definition but accepted by GPT-5.6-sol based on the actual convolution implementation. A single uncalibrated LLM judge should therefore not be treated as ground truth.

5.6 Task-level difficulty

Sixteen tasks are stable successes across every generation condition, every available output, and all three judges: `bracket_matching`, `bst_insert`, `bst_search`, `bst_zig`, `bst_zigzag`, `bst_zigzig`, `integer_exponential`, `knapsack_01`, `llrbt_flipcolor`, `llrbt_rotateleft`, `matrix_multiplication`, `queue_dequeue`, `queue_enqueue`, `ringbuffer_enqueue`, `stack_pop`, and `stack_push`.

At the other extreme, under 5.6-T0 no condition passes eight of the 65 controlled tasks: `ac_automata`, `dijkstra`, `edmond_karp`, `kruskal`, `llrbt_delete`, `longest_palindrome_substring`, `push_relabel`, and `scc_tarjan`. These concentrate graph invariants, balanced-tree preservation, and algorithm-identity requirements that are easy to replace with extensional specification solvers.

6 Failure Analysis

6.1 Compilation failures

The small number of final compilation failures fall into four distinct classes.

1. **Malformed immutable scaffold.** `trie_search` fails identically for GPT-5.5, GPT-5.6-sol, and no-thinking Opus because the teacher file itself contains a duplicate declaration and undefined identifier (Section 7.1).
2. **Response-schema failure.** GPT-5.6-sol’s final `bellman_ford`, `kruskal`, and `llrbt_delete` responses lack the required code/proof marker sections. The parser therefore has no candidate to compile.
3. **Unresolved Lean proof errors.** GPT-5.5’s `trie_insert` ends with tactic and recursion-depth errors. GPT-5.6-sol’s `llrbt_insert` leaves type mismatches and unsolved goals, while `prim` references an undefined helper theorem.
4. **Prohibited generated placeholder.** The enhanced agent’s two failures, `dijkstra` and `kruskal`, reach final candidate validation with an editable proof section still containing `sorry`. The harness correctly records these attempts as failures rather than silently omitting files.

The common compilation problem is therefore not lack of machine resources. It is a mixture of benchmark validity, output-protocol compliance, and unfinished proof construction. More repair rounds cannot solve immutable scaffolding, and schema-aware constrained decoding may be more useful than additional proof search for missing marker regions.

6.2 Why compiler-accepted programs fail semantically

Reviewing the judge analyses and generated Lean reveals five recurring patterns.

Different algorithm, correct extensional behavior. Many answers prove the supplied input–output property with a simpler algorithm: insertion or selection behavior for a named sort, direct convolution for Karatsuba, naive substring search for KMP, mutual reachability for Tarjan SCC, or exhaustive shortest paths instead of Dijkstra. Lean correctly verifies the theorem, but the answer does not satisfy the natural-language algorithm constraint.

Existing verified primitive. Examples include computing GCD with `Nat.gcd` and marking primes with `decide (Nat.Prime i)` rather than implementing trial division or a sieve. These are mathematically valid terms and useful Lean programming patterns, but they violate the benchmark’s from-scratch policy.

Brute force or nonconstructive specification oracle. Several answers enumerate all binary vectors for Gaussian elimination, all edge subsets for matching, or all flow tables for maximum flow. Others use `Classical.choose` or finite search to obtain an object known to satisfy the postcondition. This behavior exposes contracts that specify *what* result is correct but cannot encode *which* algorithm computed it.

Guarded fallback. The enhanced agent often implements a plausible target algorithm but protects it with a verified reference solver. The exported function returns the candidate only if a certificate succeeds; otherwise it rebuilds or brute-forces a correct answer. Such fallbacks maximize theorem completion while transferring correctness away from the named algorithm.

Placeholder or weak helper. An unused helper defined as 0 or `True`, or a proof that exploits a weak/vacuous precondition, may compile without providing the intended development. The judges differ on whether an unused placeholder alone warrants rejection, contributing to the disagreement in Section 4.

6.3 Residual coding-agent failures

The two temperature-zero judges agree on eight semantic rejections among the 63 compiled agent outputs:

- `ac_automata`: dynamic suffix search rather than an explicit trie with BFS-computed failure links;
- `edmond_karp`: a nonconstructive maximum-flow fallback using classical choice;
- `llrbt_delete`: validate the intended deletion, otherwise rebuild from a filtered inorder list;
- `llrbt_insert`: flatten, insert into a sorted list, and rebuild instead of local LLRBT insertion;
- `longest_palindrome_substring`: guard a Manacher-style candidate with exhaustive search;
- `max_matching`: enumerate all edge subsets instead of using augmenting paths;
- `push_relabel`: fall back to exhaustive enumeration of bounded flows;
- `scc_tarjan`: compute mutual-reachability classes without Tarjan’s stack, indices, or low-link values.

These failures persist despite explicit prompt prohibitions on exactly these strategies. Compiler interaction substantially improves theorem completion, but verifier feedback alone rewards contract satisfaction rather than algorithm identity. A semantic self-audit reduces the problem; it cannot provide a machine-checkable enforcement mechanism.

7 Benchmark and Evaluator Audit

7.1 Malformed `trie_search` scaffold

The supplied file declares `is_valid_key` twice at lines 59 and 62. It then uses an undefined `well_formed` at lines 69 and 79. These locations are outside all four model-editable sections. Direct compilation of otherwise different generated answers consistently reports:

```
line 62: 'is_valid_key' has already been declared
line 69: Unknown identifier 'well_formed'
line 79: Unknown identifier 'well_formed'
```

The generation prompt simultaneously requires all non-editable content to remain unchanged, so no conforming answer can compile. This is a benchmark defect, not a model failure. It lowers end-to-end compilation for every complete full-scope condition by one and should be repaired or removed before model comparison.

7.2 Teacher-permitted `sorry` versus semantic rejection

The generation prompt expressly warns that some `sorry` terms occur outside the editable sections, mostly in `decreasing_by` termination proofs, and instructs the model not to change them. The response parser likewise checks prohibited terms only in model-owned regions. In contrast, the semantic prompt lists visible `sorry` as a cheating signal and receives the complete merged file without machine-readable ownership annotations.

Twelve tasks have teacher-owned `sorry`: `segmenttree_build`, `segmenttree_modify`, `segmenttree_query`, `splaytree_splay`, the three ternary-search-tree tasks, the three trie tasks, `unionfind_find`, and `unionfind_linkroots`. Across the four direct conditions, 34 returned answers on this subset compile. GPT-5.4 assigns zero semantic passes at both temperatures. Its analysis of GPT-5.6-sol’s `ternarysearchtree_delete`, for example, says the delete and eager pruning are correct but rejects the answer solely because teacher definitions contain termination `sorry`.

GPT-5.6-sol at temperature zero accepts one of the same 34 answers: that exact `ternarysearchtree_delete`. Its analysis correctly distinguishes the two teacher-level termination holes from the student’s complete implementation and proof. This isolated acceptance confirms that ownership-aware adjudication is possible, but the prompt does not reliably elicit it. Scores on these 12 tasks conflate student quality with teacher scaffold policy.

The clean fix is not to allow generated proof holes. It is to give the judge only the student-owned sections plus necessary read-only context, or to annotate ownership and state explicitly that pre-existing teacher holes are

permitted. The enhanced 65-task condition excludes the confounded tasks instead of retroactively changing their labels.

7.3 Revision-prompt asymmetry

The initial generation prompt emphasizes the exact natural-language algorithm and prohibits bypasses. The subsequent Lean revision prompt says only to correct compiler messages, provide a complete standalone file, and retain marker structure. It does not restate algorithm identity, from-scratch implementation, or the distinction between teacher and student `sorry`. The original system message remains in chat, so this is an attentional asymmetry rather than a logical permission to cheat. Nevertheless, each repair turn makes compiler acceptance the immediate objective, while the later semantic judge applies the stricter algorithmic criterion.

The observed failures are consistent with this incentive: extensive repairs converge to classical choice, exhaustive solvers, alternate algorithms, or guarded fallbacks. The enhanced agent restates semantic constraints during both initial and repair phases and performs substantially better, although tool access and budget confound a causal attribution to prompt design alone. A controlled ablation is needed to measure the prompt effect.

7.4 Metric and reporting implications

The term “success rate of Lean” is ambiguous. This audit distinguishes at least: output coverage, parse compliance, compiler verification, absence of prohibited student terms, algorithmic fidelity, and judge agreement. A single number hides where the system failed and can reverse rankings when missing outputs or teacher-owned holes are handled differently. We recommend a structured result record rather than a binary leaderboard field.

8 Recommendations

1. **Preflight every scaffold.** Compile a teacher-filled reference after erasing only the four student holes; reject duplicate declarations, missing identifiers, and version drift before release.
2. **Make source ownership explicit.** Preserve marker provenance through semantic evaluation. Judges should never attribute immutable teacher terms to the student.
3. **Align all repair prompts.** Restate the exact-algorithm, from-scratch, and no-oracle constraints whenever compiler feedback is supplied.
4. **Separate formal and semantic metrics.** Report output, parse, compilation, prohibited-term validation, and semantic fidelity independently.
5. **Calibrate semantic evaluation.** Use temperature-zero runs, at least two judges or blinded human adjudication for disagreements, and publish the judge model, API mode, prompt, and sampling parameters.
6. **Test algorithm identity where possible.** Add structural obligations, trace properties, complexity/resource checks, or required intermediate invariants so that a brute-force oracle cannot satisfy the same formal contract.
7. **Record every attempt.** Empty responses, quota errors, overloads, parser failures, and prohibited-term candidates should produce durable failure artifacts rather than disappearing from the denominator.
8. **Evaluate agents as a separate treatment.** Publish tool policy, compiler-call counts, budgets, final independent verification, and costs; do not merge agentic scores with direct prompting.

9 Threats to Validity

Single samples. Each generation condition contains one pass per task. We do not estimate variance over random seeds, API nondeterminism, or temporal model updates. Temperature zero is a sampling setting, not a guarantee of bitwise determinism.

LLM semantic labels. No expert manually relabeled all 373 condition–task pairs. The judge analyses are useful evidence, not formal proofs of algorithm identity. The measured 22 cross-judge flips demonstrate residual label uncertainty.

Unequal treatments. Provider reasoning controls and token accounting are not directly comparable. The coding agent receives tools, more interaction, and a monetary budget. Its result is an upper-level system comparison, not an isolated Opus model effect.

Adaptive-thinking attrition. The 49 returned adaptive-thinking outputs are not a random subset of 77. Conditional quality on those outputs is subject to selection bias; end-to-end rates retain missing outputs in the denominator.

Benchmark and version scope. Findings apply to the audited repository snapshot, Lean 4.25.0-rc2, and the named model endpoints in July–August 2026. They do not automatically generalize to corrected scaffolds, other Lean/Mathlib releases, other languages in AlgoVeri, or future endpoint revisions.

Cost accounting. Only Claude Code reports dollar cost in the saved records. Token fields differ by provider (for example, Anthropic thinking is not stored in the same field as OpenAI reasoning), so we release raw counters but avoid a cross-provider efficiency ranking.

10 Reproducibility and Artifact Layout

The artifact repository is <https://github.com/Joey-the-Creater/AlgoVeri-Rerun>. It includes generation JSON, semantic JSON for all three judge settings, source modifications, tests, the dashboard, and this report. The principal immutable directories are:

- `results/full_three`: GPT-5.5, GPT-5.6-sol, and thinking-Opus generation;
- `results/full_opus_no_thinking`: no-thinking Opus generation;
- `results/claude_code_opus5`: enhanced 65-task agent generation;
- `matching_semantic`, `_semantic_temp0`, and `_semantic_gpt56_temp0` directories for the three judges;
- `reports/data/result_summary.json` and `reports/data/case_results.csv`: recomputed aggregate and case-level data;
- `config/dashboard_experiments.json`: the canonical experiment catalog.

From the repository root, regenerate the analytical artifacts with:

```
python3 scripts/analyze_rerun_results.py \  
--json-output reports/data/result_summary.json \  
--csv-output reports/data/case_results.csv \  
--latex-output reports/generated/case_tables.tex
```

Compile the paper locally or in Overleaf with:

```
cd reports  
latexmk -pdf algoveri_rerun_report.tex
```

The CSV contains every judge analysis in full. The PDF matrix uses compact pass/fail codes to remain readable; the underlying generated Lean and complete conversations are preserved in the result JSON.

11 Conclusion

Current frontier models make AlgoVeri’s Lean files compile at high rates, but compilation substantially overstates success on the named algorithm. No-thinking Opus is the strongest complete direct condition in this rerun, while a constrained compiler-using agent raises semantic acceptance further on a clean common scope. Its remaining failures show that verifier-guided search still gravitates toward extensionally correct oracles and fallbacks when algorithm identity is not machine-enforced.

At the same time, the benchmark itself introduces avoidable error: one task is uncompileable under its editing policy, and 12 tasks create a prompt-level conflict over teacher-owned proof holes. Semantic scores also change with judge model despite temperature zero. The appropriate conclusion is therefore not merely a model ranking. Reliable verified-code evaluation requires valid scaffolds, explicit authorship, aligned repair prompts, durable failure records, independent compilation, and calibrated semantic adjudication.

A Native-Scope Failure Inventories

GPT-5.5 compilation failures (2). `trie_insert`, `trie_search`.

GPT-5.6-sol compilation failures (6). `bellman_ford`, `kruskal`, `llrbt_delete`, `llrbt_insert`, `prim`, `trie_search`.

No-thinking Opus compilation failures (1). `trie_search`.

Enhanced Claude Code compilation failures (2). `dijkstra`, `kruskal`.

Thinking-Opus missing outputs (28). `bellman_ford`, `bfs`, `coin_change`, `cycle_detection`, `dfs`, `dijkstra`, `jump_game`, `knapsack_unbounded`, `kruskal`, `lca`, `llrbt_delete`, `llrbt_insert`, `maxheap_popmax`, `prim`, `push_relabel`, `scc_tarjan`, `segmenttree_modify`, `segmenttree_query`, `ternarysearchtree_delete`, `ternarysearchtree_insert`, `ternarysearchtree_search`, `topological_sort`, `trial_division_optimized`, `trie_delete`, `trie_insert`, `trie_search`, `unionfind_find`, `unionfind_linkroots`.

B Complete Case Matrix and Judge Disagreements

Table 5: Complete case-level outcome matrix. A cell is C:abc when Lean compiles; a, b, and c are semantic outcomes for GPT-5.4 at temperature 1, GPT-5.4 at temperature 0, and GPT-5.6-sol at temperature 0, respectively (P=pass, F=fail). X denotes a saved output that does not compile, M a missing output, and – an out-of-scope case.

Task	GPT-5.5	GPT-5.6-sol	Opus think	Opus no-think	Claude Code
ac_automata	C:FFF	C:FFF	C:FFF	C:FFF	C:FFF
bellman_ford	C:FFF	X	M	C:FPP	C:PPP
bfs	C:FFF	C:FFF	M	C:FFF	C:PPP
binary_search	C:FFF	C:PPP	C:PPP	C:PPP	C:PPP
bipartite_check	C:FFF	C:FPP	C:FPP	C:FPP	C:PPP
bracket_matching	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
bst_delete	C:PPP	C:FFF	C:PPP	C:PPP	C:PPP
bst_insert	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
bst_search	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
bst_zig	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
bst_zigzag	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
bst_zigzig	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
bubble_sort	C:FFF	C:FFF	C:PPP	C:PPP	C:FPP
coin_change	C:FFF	C:FFF	M	C:PPP	C:PPP
cycle_detection	C:FFF	C:FFF	M	C:FPP	C:PPP
dfs	C:FFF	C:FFF	M	C:FFF	C:PPP
dijkstra	C:FFF	C:FFF	M	C:FFF	X
discrete_logarithm	C:FFF	C:PPP	C:FFF	C:PPP	C:PPP
edmond_karp	C:FFF	C:FFF	C:FFF	C:FFF	C:FFF
fast_exponential	C:FFF	C:FPP	C:PPF	C:FPP	C:PPP
gcd	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
house_robber	C:FFF	C:FFF	C:FFF	C:PPP	C:PPP
insertion_sort	C:FFF	C:PPP	C:PPP	C:FFF	C:PPP
integer_exponential	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
jump_game	C:FFF	C:FFF	M	C:FPP	C:PPP
k_smallest	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
kmp	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
knapsack_01	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
knapsack_unbounded	C:PFF	C:FFF	M	C:PPP	C:PPP
kruskal	C:FFF	X	M	C:FFF	X
lca	C:PFP	C:FPP	M	C:PPP	C:PPP
linear_search	C:PPP	C:PPP	C:FFF	C:PPP	C:PPP
linearsys_gf2	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
llrbt_delete	C:FFF	X	M	C:FFF	C:FFF
llrbt_flipcolor	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
llrbt_insert	C:FFF	X	M	C:PPP	C:FFF
llrbt_rotateleft	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
llrbt_rotateright	C:FFF	C:FPP	C:FPP	C:FPP	C:FPP
longest_common_subsequence	C:PPP	C:PPP	C:FPP	C:PPP	C:PPP
longest_increasing_subsequence	C:PPF	C:PPF	C:PPP	C:PPP	C:PPP
longest_palindrome_substring	C:FFF	C:FFF	C:FFF	C:FFF	C:FFF
matrix_multiplication	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
max_matching	C:FFF	C:FFF	C:FPP	C:FPP	C:FFF

Continued on next page

Table 5 – continued

Task	GPT-5.5	GPT-5.6-sol	Opus think	Opus no-think	Claude Code
maxheap_popmax	C:FFF	C:FFF	M	C:FFF	C:PPP
maxheap_push	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
maximum_subarray_sum	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
merge_sort	C:FFF	C:FFF	C:PPP	C:PPP	C:PPP
polymul_karatsuba	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
polymul_naive	C:PPP	C:FFP	C:PFF	C:FFP	C:FFP
prim	C:FFF	X	M	C:FFF	C:PPP
push_relabel	C:FFF	C:FFF	M	C:FFF	C:FFF
queue_dequeue	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
queue_enqueue	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
quick_sort	C:FFF	C:PPP	C:PPP	C:PPP	C:PPP
ringbuffer_dequeue	C:FFP	C:FFP	C:PPP	C:PPP	C:PPP
ringbuffer_enqueue	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
rod_cutting	C:PFF	C:PPP	C:PPP	C:PPP	C:PPP
scc_tarjan	C:FFF	C:FFF	M	C:FFF	C:FFF
segmenttree_build	C:FFF	C:FFF	C:FFF	C:FFF	–
segmenttree_modify	C:FFF	C:FFF	M	C:FFF	–
segmenttree_query	C:FFF	C:FFF	M	C:FFF	–
sieve_method	C:FFF	C:FFF	C:FFF	C:FFF	C:PPP
splaytree_splay	C:FFF	C:FFF	C:FFF	C:FFF	–
stack_pop	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
stack_push	C:PPP	C:PPP	C:PPP	C:PPP	C:PPP
string_search_naive	C:PPP	C:PPF	C:PPP	C:PPP	C:PPP
ternarysearchtree_delete	C:FFF	C:FFP	M	C:FFF	–
ternarysearchtree_insert	C:FFF	C:FFF	M	C:FFF	–
ternarysearchtree_search	C:FFF	C:FFF	M	C:FFF	–
topological_sort	C:FFF	C:FFF	M	C:PPP	C:PPP
trial_division_naive	C:FPP	C:FFF	C:PPP	C:PPP	C:PPP
trial_division_optimized	C:FFF	C:FFF	M	C:PPP	C:PPP
trie_delete	C:FFF	C:FFF	M	C:FFF	–
trie_insert	X	C:FFF	M	C:FFF	–
trie_search	X	X	M	X	–
unionfind_find	C:FFF	C:FFF	M	C:FFF	–
unionfind_linkroots	C:FFF	C:FFF	M	C:FFF	–

Table 6: All semantic-judge disagreements on compiler-accepted outputs.

Generation	Comparison	n	Agree	κ	Flips	Disagreement cases
GPT-5.5	5.4 T1 vs 5.4 T0	75	71	0.877	4	knapsack_unbounded (L), lca (L), rod_cutting (R), trial_division_naive (R)
GPT-5.5	5.4 T0 vs 5.6 T0	75	71	0.877	4	lca (R), longest_increasing_subsequence (L), ringbuffer_dequeue (R), rod_cutting (L)
GPT-5.6-sol	5.4 T1 vs 5.4 T0	71	69	0.939	2	lca (R), ringbuffer_dequeue (R)
GPT-5.6-sol	5.4 T0 vs 5.6 T0	71	64	0.795	7	bipartite_check (R), fast_exponential (R), llrbt_rotateright (R), longest_increasing_subsequence (L), polymul_naive (R), string_search_naive (L), ternarysearchtree_delete (R)
Opus 5 (thinking)	5.4 T1 vs 5.4 T0	49	47	0.916	2	longest_common_subsequence (R), polymul_naive (L)
Opus 5 (thinking)	5.4 T0 vs 5.6 T0	49	45	0.828	4	bipartite_check (R), fast_exponential (L), llrbt_rotateright (R), max_matching (R)
Opus 5 (no thinking)	5.4 T1 vs 5.4 T0	76	73	0.921	3	bellman_ford (R), bipartite_check (R), jump_game (R)
Opus 5 (no thinking)	5.4 T0 vs 5.6 T0	76	71	0.868	5	cycle_detection (R), fast_exponential (R), llrbt_rotateright (R), max_matching (R), polymul_naive (R)
Claude Code + Opus 5 (enhanced)	5.4 T1 vs 5.4 T0	63	61	0.881	2	bubble_sort (L), llrbt_rotateright (R)
Claude Code + Opus 5 (enhanced)	5.4 T0 vs 5.6 T0	63	61	0.871	2	bubble_sort (R), polymul_naive (R)

References

- [1] Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, Lacramioara Astefanoaei, Ionel Emilian Chiosa, Jure Kukovec, Alok Singh, Oliver Butterley, Adem Bizid, Quinn Dougherty, Miranda Zhao, Max Tan, and Max Tegmark. A benchmark for vericoding: Formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025. URL <https://arxiv.org/abs/2509.22908>.
- [2] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.
- [3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [4] Joey Li. AlgoVeri-Rerun: Code, results, and reproducibility artifacts. <https://github.com/Joey-the-Creater/AlgoVeri-Rerun>, 2026.
- [5] Chloe Loughridge, Qinyi Sun, Seth Ahrenbach, Federico Cassano, Chuyue Sun, Ying Sheng, Anish Mudide, Md Rakib Hossain Misu, Nada Amin, and Max Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024. URL <https://arxiv.org/abs/2406.08467>.
- [6] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- [7] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.
- [8] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [9] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.15626>.
- [10] Haoyu Zhao and contributors. AlgoVeri source repository. <https://github.com/haoyuzhao123/algoveri>, 2026. Accessed 2026-08-06.
- [11] Haoyu Zhao, Ziran Yang, Jiawei Li, Deyuan He, Zenan Li, Chi Jin, Venugopal V. Veeravalli, Aarti Gupta, and Sanjeev Arora. AlgoVeri: An aligned benchmark for verified code generation on classical algorithms. *arXiv preprint arXiv:2602.09464*, 2026. URL <https://arxiv.org/abs/2602.09464>.
- [12] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2109.00110>.
- [13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.05685>.